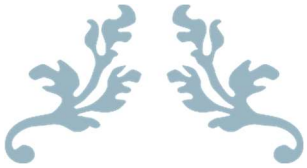


RPN CALCULATOR EXAMPLES

Examples and Visualizations

FEBRUARY 3, 2026

Contents

Example 1: Basic Arithmetic Operations

This example demonstrates fundamental arithmetic operations using Reverse Polish Notation (RPN). In RPN, operators follow their operands, eliminating the need for parentheses.

Code Example

```
from rpn_calculator import Calculator

calc = Calculator(enable_logging=False)

# Addition: 3 + 4
result = calc.evaluate_and_clear('3 4 +')
print(result)  # 7

# Power: 2^8
result = calc.evaluate_and_clear('2 8 ^')
print(result)  # 256

# Complex: 5 * (2 + 3)
result = calc.evaluate_and_clear('5 2 3 + *')
print(result)  # 25
```

Sample Output

Description	RPN Expression	Result
Addition	`3 4 +`	7
Subtraction	`10 3 -`	7
Power	`2 8 ^`	256
Complex Expression	`5 2 3 + *`	25

Example 2: Trigonometric Functions

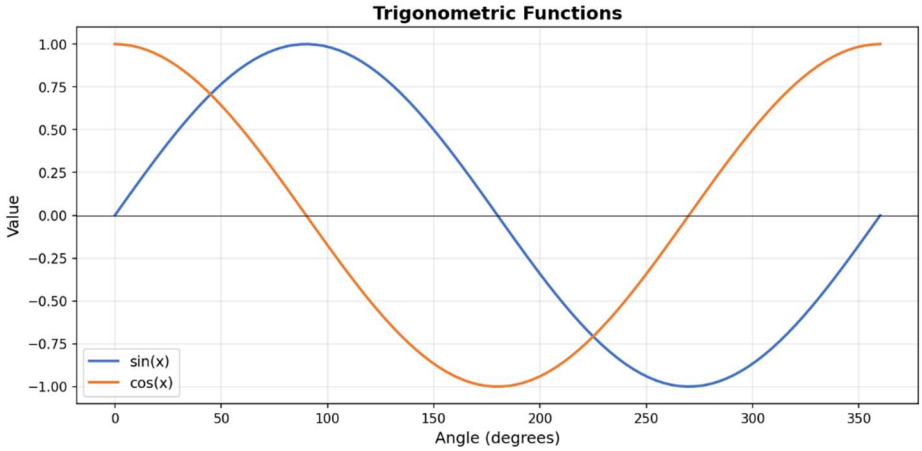
The RPN calculator supports all standard trigonometric functions (SIN, COS, TAN, ASIN, ACOS, ATAN) and respects the angle mode setting (degrees or radians).

Code Example

```
# Set to degrees mode
calc.state.degrees = True

# Calculate sin(30°)
result = calc.evaluate_and_clear('30 SIN')
print(result)  # 0.5

# Calculate cos(60°)
result = calc.evaluate_and_clear('60 COS')
print(result)  # 0.5
```



Example 3: Complex Number Operations

Complex numbers can be created in rectangular (a+bi) or polar (r∠θ) form. The calculator provides full support for complex arithmetic and conversions.

Code Example

```
# Create complex number 3+4i
calc.evaluate_and_clear('3 4 CMPLX')
z = calc.stack[-1]
print(z)  # (3+4j)

# Get magnitude
calc.evaluate_and_clear('3 4 CMPLX ABS')
print(calc.stack[-1])  # 5.0

# Convert polar to rectangular
calc.evaluate_and_clear('5 53.13 RECT')
print(calc.stack[-1])  # ~(3+4j)
```

Key Operations

- **CMPLX** - Create from real and imaginary parts
- **RECT** - Convert polar (r, θ) to rectangular
- **POLAR** - Convert rectangular to polar
- **ABS** - Magnitude (modulus)
- **ARG** - Phase angle
- **CONJ** - Complex conjugate

Example 4: Vector Operations

The calculator supports vector operations including dot product, cross product, magnitude, and normalization. Vectors are represented as Python lists.

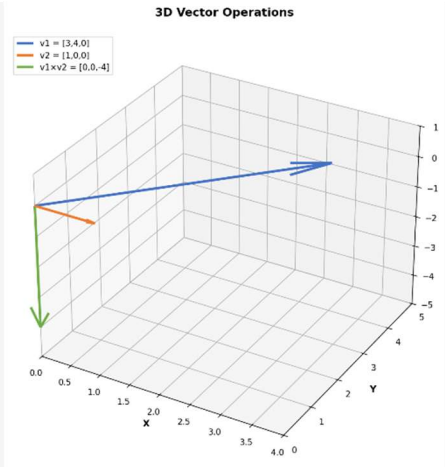
Code Example

```
# Define vectors
v1 = [3, 4, 0]
v2 = [1, 0, 0]

# Dot product
calc.push(v1)
calc.push(v2)
calc.evaluate('DOT')
print(calc.pop())  # 3

# Magnitude
calc.push(v1)
calc.evaluate('VMAG')
print(calc.pop())  # 5.0

# Cross product (3D only)
calc.push(v1)
calc.push(v2)
calc.evaluate('VCROSS')
print(calc.pop())  # [0, 0, -4]
```



Example 5: FFT and Signal Processing

Fast Fourier Transform (FFT) operations enable frequency domain analysis of signals. The calculator automatically zero-pads signals to the next power of 2 for optimal performance.

Code Example

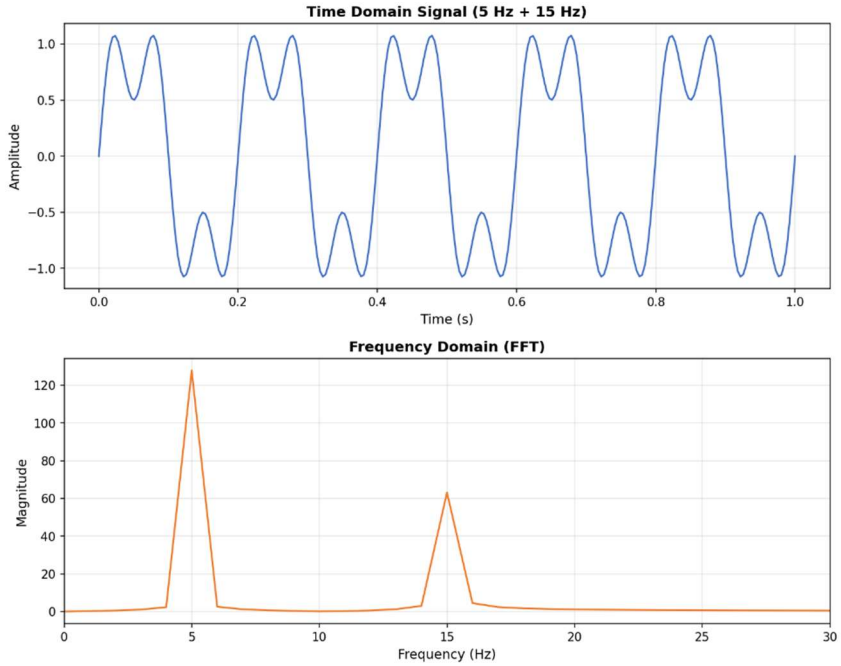
```
import numpy as np

# Create signal (5 Hz + 15 Hz)
t = np.linspace(0, 1, 256)
signal = np.sin(2*np.pi*5*t) + 0.5*np.sin(2*np.pi*15*t)

# Compute FFT
calc.push(signal.tolist())
calc.evaluate('FFT_MAG')
spectrum = calc.pop()
```

Available FFT Operations

- **FFT** - Forward FFT (returns complex values)
- **IFFT** - Inverse FFT
- **FFT_MAG** - Magnitude spectrum
- **FFT_PHASE** - Phase spectrum



Example 6: Matrix Operations

The calculator provides comprehensive matrix operations including arithmetic, decompositions, and eigenvalue analysis. Matrices are represented as lists of lists.

Code Example

```
# Define matrices
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

# Matrix multiplication
calc.push(A)
calc.push(B)
calc.evaluate('M*')
result = calc.pop()
# [[19, 22], [43, 50]]

# Determinant
calc.push(A)
calc.evaluate('DET')
print(calc.pop())  # -2

# Inverse
calc.push(A)
calc.evaluate('MINV')
inverse = calc.pop()
```

Matrix Operations

Operation	Description
`M+, M-, M*`	Matrix addition, subtraction, multiplication
`DET`	Determinant
`MINV`	Matrix inverse
`MTRANS`	Transpose
`EIGEN`	Eigenvalues and eigenvectors
`LU, QR, SVD`	Matrix decompositions

Example 7: Statistical Operations

```
  # Standard deviation
    calc.push(data)
    calc.evaluate("STDV")
    stdv = calc.pop()
    print(f"\nStandard Deviation: {stdv:.4f}")

    # Mean (using vector operations)
    total = sum(data)
    mean = total / len(data)
    print(f"Mean: {mean:.4f}")

    # Combinations
    calc.evaluate_and_clear("10 3 COMB")
    comb = calc.stack[-1]
    print(f"\nC(10,3) = {comb}")

    # Permutations
    calc.evaluate_and_clear("10 3 PERM")
    perm = calc.stack[-1]
    print(f"P(10,3) = {perm}")
```

Statistical Operations

Operation	Description
COMB	Combinations
PERM	Permutations
STDV	Standard Deviation

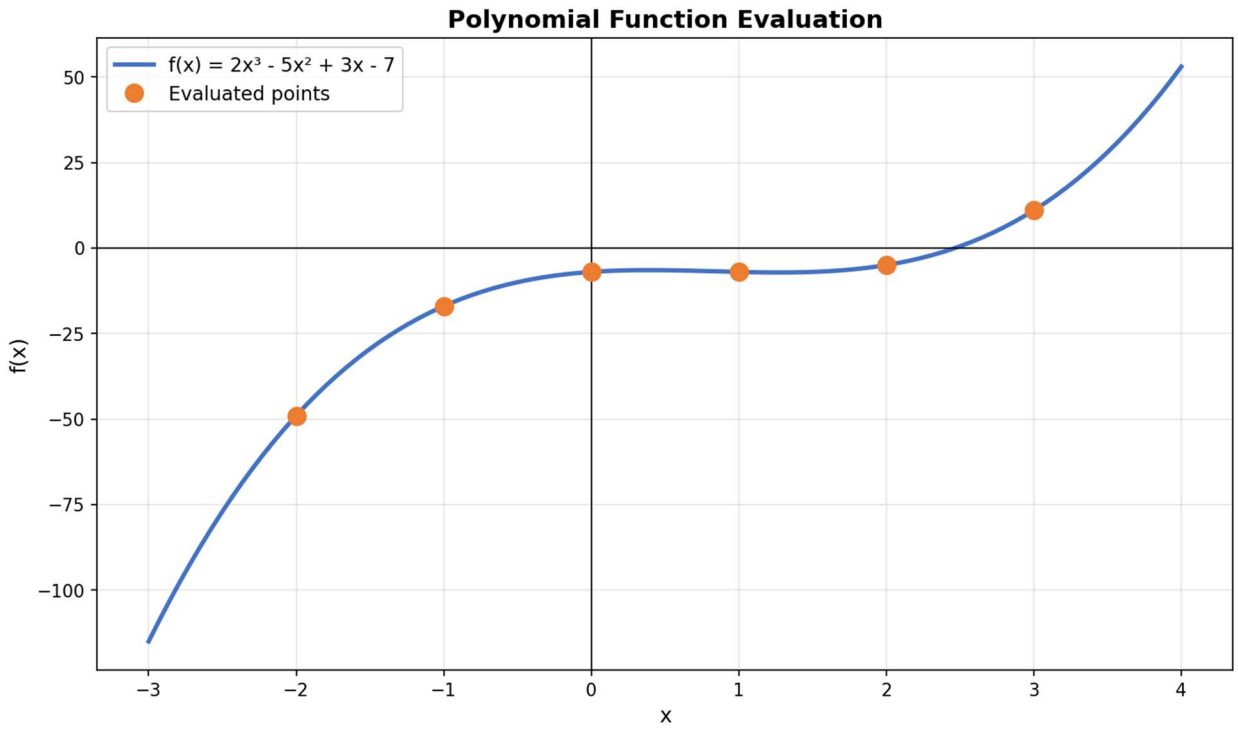
Example 8: Polynomial Evaluation

```
    # Evaluate polynomial: f(x) = 2x^3 - 5x^2 + 3x - 7
    # In RPN: x 3 ^ 2 * x 2 ^ 5 * - x 3 * + 7 -

    print("Polynomial: f(x) = 2x³ - 5x² + 3x - 7")
    print("\nEvaluations:")

    x_vals = [-2, -1, 0, 1, 2, 3]
    y_vals = []

    for x in x_vals:
        expr = f"{x} 3 ^ 2 * {x} 2 ^ 5 * - {x} 3 * + 7 -"
        result = calc.evaluate_and_clear(expr)
        y_vals.append(result)
        print(f"  f({x:2}) = {result:8.2f}")
```



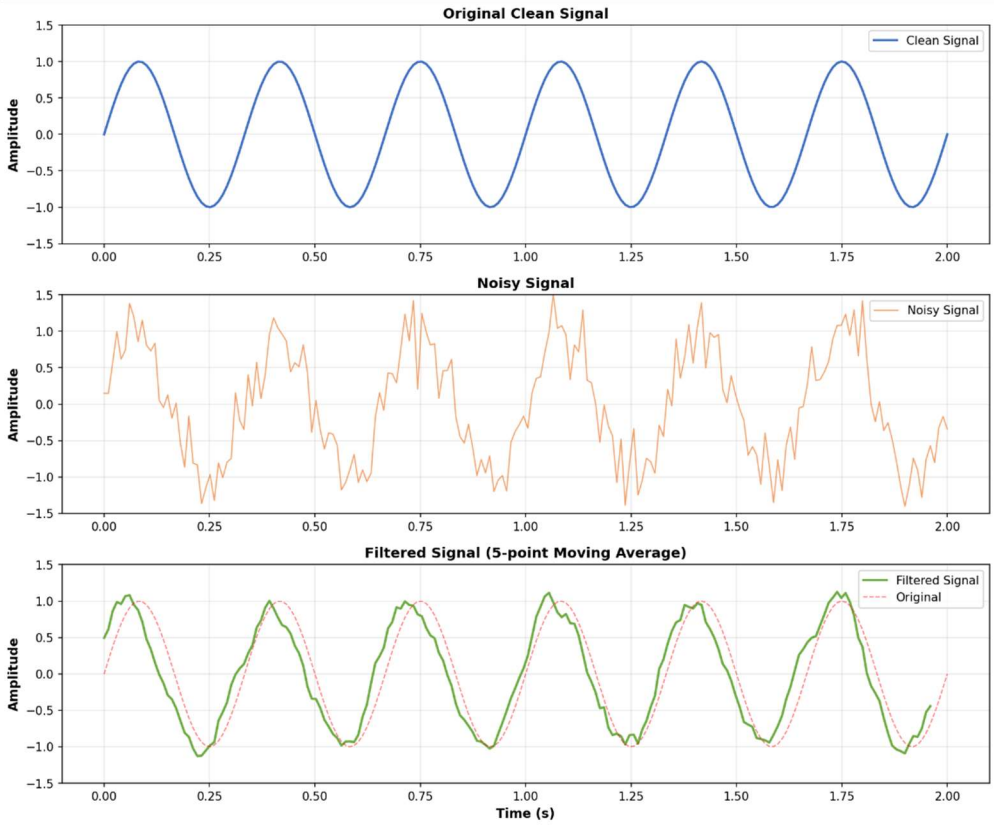
Example 9: Signal Filtering

```
# Create noisy signal
    np.random.seed(42)
    t = np.linspace(0, 2, 200)
    clean_signal = np.sin(2 * np.pi * 3 * t)
    noise = 0.3 * np.random.randn(len(t))
    noisy_signal = clean_signal + noise

    # Simple moving average filter (kernel)
    kernel_size = 5
    kernel = [1/kernel_size] * kernel_size

    print(f"Signal length: {len(noisy_signal)}")
    print(f"Filter kernel: {kernel_size}-point moving average")

    # Apply convolution
    calc.push(noisy_signal.tolist())
    calc.push(kernel)
    calc.evaluate("CONV")
    filtered = calc.pop()
```



Example 10: CSV Data Processing

One of the most practical uses of the RPN calculator is processing tabular data from CSV files. This example shows electrical circuit analysis.

Code Example

```
import csv

calc = Calculator(enable_logging=False)

with open('data.csv', 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        v = float(row['Voltage'])
        i = float(row['Current'])

        # Power: P = V * I
        power = calc.evaluate_and_clear(f'{v} {i} *')

        # Resistance: R = V / I
        resistance = calc.evaluate_and_clear(f'{v} {i} /')
```

Sample Data and Results

Voltage (V)	Current (A)	Power (W)	Resistance (Ω)
5.0	0.5	2.5	10.0
10.0	1.0	10.0	10.0
20.0	2.0	40.0	10.0

Best Practices & Tips

Library Integration

- **Disable logging:** Always use `Calculator(enable_logging=False)` when using as a library to avoid file I/O overhead
- **Use evaluate_and_clear:** For independent calculations, use `evaluate_and_clear()` to prevent stack accumulation
- **Handle errors:** Always wrap calculator operations in try-except blocks to catch `CalculatorError` exceptions
- **Configure state:** Set `calc.state.degrees` and `calc.state.format` before trigonometric operations

RPN Expression Tips

- **Operators follow operands:** In RPN, `3 4 +` means 3 + 4, not + 3 4
- **No parentheses needed:** Complex expressions like (3+4)×5 become `3 4 + 5 *`
- **Vector notation:** Use Python list syntax: `[1,2,3]` for vectors
- **Matrix notation:** Use nested lists: `[[1,2],[3,4]]` for 2×2 matrices

Performance Considerations

- **FFT auto-pads:** FFT operations automatically zero-pad to the next power of 2 for optimal performance
- **Batch processing:** For large datasets, process in batches and reuse the calculator instance
- **Direct stack access:** For complex workflows, use `calc.push()` and `calc.pop()` for fine-grained control

Appendix: Complete Operation Reference

This appendix provides a comprehensive list of all operations supported by the RPN calculator, organized by category.

Arithmetic Operations
```
+ - * / ^ MOD ||
```

Trigonometric Functions
```
SIN COS TAN ASIN ACOS ATAN
```

Logarithmic Functions
```
LOG LN EXP SQRT 1/X
```

Complex Number Operations
```
CMPLX RECT POLAR RE IM ABS ARG CONJ
```

Vector Operations
```
DOT VMAG VCROSS VNORM
```

Matrix Operations
```
MATRIX DET TRACE MINV M+ M- M* MTRANS
```

Matrix Decompositions
```
LU QR SVD CHOLESKY SCHUR EIGEN
```

FFT & Signal Processing
```
FFT IFFT FFT_MAG FFT_PHASE CONV CONV2 XCORR
```

Statistics & Combinatorics10
```
COMB PERM STDV GCD LCM
```

Stack Operations
```
C DEL UNDO SWAP RD RU
```

Constants
```
PI E I
```

Appendix: Error Handling Examples

The following section provides examples of error handling.based on rpn_calculator.py


```python
#!/usr/bin/env python3
"""
RPN Calculator - Error Handling Examples
Demonstrates proper error handling patterns for robust applications
"""
from rpn_calculator import Calculator, CalculatorError


def example_basic_error_handling():
    """Example 1: Basic Error Handling Pattern"""
    print("=" * 70)
    print("EXAMPLE 1: Basic Error Handling")
    print("=" * 70)

    calc = Calculator(enable_logging=False)

    # Try a division by zero
    try:
        result = calc.evaluate_and_clear("10 0 /")
        print(f"Result: {result}")
    except CalculatorError as e:
        print(f"❌ Error caught: {e.message}")
        print("✓ Application continues running safely")

    print()


def example_invalid_operations():
    """Example 2: Handling Invalid Operations"""
    print("=" * 70)
    print("EXAMPLE 2: Invalid Operations")
```

```
    print("=" * 70)

    calc = Calculator(enable_logging=False)

    # Not enough operands
    try:
        result = calc.evaluate_and_clear("5 +")  # Missing second
operand
        print(f"Result: {result}")
    except CalculatorError as e:
        print(f"❌ Not enough operands: {e.message}")

    # Invalid token
    try:
        result = calc.evaluate_and_clear("5 INVALID_OP")
        print(f"Result: {result}")
    except CalculatorError as e:
        print(f"❌ Invalid operation: {e.message}")

    print()


def example_csv_processing_with_errors():
    """Example 3: CSV Processing with Robust Error Handling"""
    print("=" * 70)
    print("EXAMPLE 3: CSV Processing with Error Handling")
    print("=" * 70)

    import csv
    import os
    import tempfile

    # Create sample CSV with some problematic data
    # Use Windows-compatible temp directory
    temp_dir = tempfile.gettempdir()
    csv_file = os.path.join(temp_dir, 'sample_with_errors.csv')

    csv_data = [
        ['x', 'y', 'operation'],
        ['10', '2', 'x y /'],        # Valid: 10/2 = 5
        ['5', '0', 'x y /'],          # Error: division by zero
        ['3', '4', 'x y + 2 /'],      # Valid: (3+4)/2 = 3.5
        ['abc', '5', 'x y +'],        # Error: invalid number
        ['8', '2', 'x y ^ SQRT'],     # Valid: sqrt(8^2) = 8
    ]

    with open(csv_file, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerows(csv_data)

    print(f"Processing CSV file: {csv_file}")
    print()

    calc = Calculator(enable_logging=False)
    results = []
```

```
        error_count = 0
        success_count = 0

        with open(csv_file, 'r') as f:
            reader = csv.DictReader(f)

            for row_num, row in enumerate(reader, start=2):  # Start at 2 (row 1 is header)
                x_str = row['x']
                y_str = row['y']
                operation = row['operation']

                # Replace placeholders with actual values
                expr = operation.replace('x', x_str).replace('y', y_str)

                try:
                    # Attempt to evaluate the expression
                    result = calc.evaluate_and_clear(expr)
                    success_count += 1

                    print(f"✓ Row {row_num}: {x_str}, {y_str} → {operation} = {result:.4f}")
                    results.append({
                        'row': row_num,
                        'x': x_str,
                        'y': y_str,
                        'result': result,
                        'status': 'success'
                    })

                except CalculatorError as e:
                    error_count += 1
                    print(f"✗ Row {row_num}: {x_str}, {y_str} → ERROR: {e.message}")
                    results.append({
                        'row': row_num,
                        'x': x_str,
                        'y': y_str,
                        'result': None,
                        'status': 'error',
                        'error_message': e.message
                    })

                except ValueError as e:
                    # Handle conversion errors (invalid numbers)
                    error_count += 1
                    print(f"✗ Row {row_num}: {x_str}, {y_str} → VALUE ERROR: Invalid number format")
                    results.append({
                        'row': row_num,
                        'x': x_str,
                        'y': y_str,
                        'result': None,
                        'status': 'error',
                        'error_message': f'Invalid number: {str(e)}'
```

```
                    })

        print()
        print(f"Summary: {success_count} successful, {error_count}
errors")
        print()

        # Clean up
        if os.path.exists(csv_file):
            os.remove(csv_file)


    def example_graceful_degradation():
        """Example 4: Graceful Degradation Pattern"""
        print("=" * 70)
        print("EXAMPLE 4: Graceful Degradation")
        print("=" * 70)

        calc = Calculator(enable_logging=False)

        def safe_calculate(expression, default=None):
            """
            Safely evaluate an expression, returning default value on
error.
            This pattern is useful when you want the program to continue
            even if some calculations fail.
            """
            try:
                return calc.evaluate_and_clear(expression)
            except CalculatorError as e:
                print(f"  Warning: {e.message} - using default value:
{default}")
                return default

        # Process a batch of calculations
        calculations = [
            ("3 4 +", "sum"),
            ("10 0 /", "division by zero"),  # Will error
            ("5 2 ^", "power"),
            ("INVALID", "bad syntax"),        # Will error
            ("2 3 * 4 +", "complex"),
        ]

        results = {}
        for expr, name in calculations:
            result = safe_calculate(expr, default=0)
            results[name] = result
            if result is not None:
                print(f"✓ {name:20} = {result}")

        print()
        print(f"Processed {len(calculations)} calculations")
        print(f"Valid results: {sum(1 for v in results.values() if v is
not None)}")
        print()
```

```
 188
 189
 190  def example_retry_pattern():
 191      """Example 5: Retry Pattern with Fallback"""
 192      print("=" * 70)
 193      print("EXAMPLE 5: Retry Pattern with Fallback")
 194      print("=" * 70)
 195
 196      calc = Calculator(enable_logging=False)
 197
 198      def calculate_with_fallback(primary_expr, fallback_expr):
 199          """
 200          Try primary expression first, fall back to alternative if it
fails.
 201          Useful for trying optimal method first, with a simpler
backup.
 202          """
 203          try:
 204              result = calc.evaluate_and_clear(primary_expr)
 205              print(f"✓ Primary succeeded: {primary_expr} = {result}")
 206              return result
 207          except CalculatorError as e:
 208              print(f"  Primary failed ({e.message}), trying
fallback...")
 209              try:
 210                  result = calc.evaluate_and_clear(fallback_expr)
 211                  print(f"✓ Fallback succeeded: {fallback_expr} =
{result}")
 212                  return result
 213              except CalculatorError as e2:
 214                  print(f"✗ Both methods failed: {e2.message}")
 215                  raise
 216
 217      # Example: Try inverse, fall back to alternative calculation
 218      print("Attempting calculation with fallback:")
 219
 220      # This will work
 221      result1 = calculate_with_fallback("4 2 /", "4 2 -")
 222      print()
 223
 224      # This will use fallback (trying to invert zero matrix would
fail)
 225      # For demonstration, we'll use a simpler example
 226      result2 = calculate_with_fallback("10 0 /", "10 1 /")
 227      print()
 228
 229
 230  def example_validation_pattern():
 231      """Example 6: Input Validation Before Calculation"""
 232      print("=" * 70)
 233      print("EXAMPLE 6: Input Validation Pattern")
 234      print("=" * 70)
 235
 236      calc = Calculator(enable_logging=False)
 237
```

```python
    def validate_and_calculate(x, y, operation):
        """
        Validate inputs before attempting calculation.
        This prevents errors and provides better user feedback.
        """
        # Input validation
        errors = []

        # Check if numbers are valid
        try:
            x_val = float(x)
        except (ValueError, TypeError):
            errors.append(f"Invalid x value: '{x}'")

        try:
            y_val = float(y)
        except (ValueError, TypeError):
            errors.append(f"Invalid y value: '{y}'")

        # Check for specific problematic cases
        if operation == '/' and str(y) == '0':
            errors.append("Division by zero not allowed")

        # If validation failed, report all errors
        if errors:
            print(f"❌ Validation failed:")
            for error in errors:
                print(f"   - {error}")
            return None

        # Validation passed, perform calculation
        try:
            expr = f"{x} {y} {operation}"
            result = calc.evaluate_and_clear(expr)
            print(f"✓ {x} {operation} {y} = {result}")
            return result
        except CalculatorError as e:
            print(f"❌ Calculation error: {e.message}")
            return None

    # Test cases
    test_cases = [
        (10, 2, '+'),       # Valid
        (10, 0, '/'),       # Caught by validation
        ('abc', 5, '*'),    # Invalid input
        (8, 2, '^'),        # Valid
    ]

    for x, y, op in test_cases:
        print(f"\nTesting: {x} {op} {y}")
        validate_and_calculate(x, y, op)

    print()
```

```python
def example_logging_errors():
    """Example 7: Logging Errors for Debugging"""
    print("=" * 70)
    print("EXAMPLE 7: Error Logging Pattern")
    print("=" * 70)

    import logging
    from datetime import datetime

    # Set up logging
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s - %(levelname)s - %(message)s'
    )
    logger = logging.getLogger(__name__)

    calc = Calculator(enable_logging=False)

    def calculate_with_logging(expression, context=""):
        """
        Calculate with comprehensive error logging.
        Useful for production systems where you need audit trails.
        """
        logger.info(f"Starting calculation: '{expression}' {context}")

        try:
            result = calc.evaluate_and_clear(expression)
            logger.info(f"Success: {expression} = {result}")
            return result

        except CalculatorError as e:
            logger.error(f"CalculatorError in '{expression}': {e.message}")
            logger.debug(f"Context: {context}")
            return None

        except Exception as e:
            logger.critical(f"Unexpected error in '{expression}': {str(e)}")
            logger.debug(f"Context: {context}", exc_info=True)
            return None

    # Run calculations with logging
    calculations = [
        ("3 4 +", "User: Alice, Session: 12345"),
        ("10 0 /", "User: Bob, Session: 67890"),
        ("5 2 ^", "User: Alice, Session: 12345"),
    ]

    for expr, context in calculations:
        calculate_with_logging(expr, context)

    print()
```

```python
def example_batch_processing():
    """Example 8: Batch Processing with Error Summary"""
    print("=" * 70)
    print("EXAMPLE 8: Batch Processing with Error Summary")
    print("=" * 70)

    calc = Calculator(enable_logging=False)

    # Simulated batch of expressions
    expressions = [
        "3 4 +",
        "10 5 /",
        "2 8 ^",
        "15 0 /",       # Error
        "5 INVALID",    # Error
        "7 3 -",
        "9 SQRT",
        "0 1 /",
        "BADTOKEN",     # Error
        "12 3 MOD",
    ]

    results = []
    errors = []

    print(f"Processing {len(expressions)} expressions...\n")

    for i, expr in enumerate(expressions, 1):
        try:
            result = calc.evaluate_and_clear(expr)
            results.append({
                'index': i,
                'expression': expr,
                'result': result,
                'status': 'success'
            })
            print(f"  {i:2}. ✓ {expr:20} = {result}")

        except CalculatorError as e:
            errors.append({
                'index': i,
                'expression': expr,
                'error': e.message,
                'status': 'failed'
            })
            print(f"  {i:2}. ✗ {expr:20} → {e.message}")

    # Summary report
    print()
    print("=" * 70)
    print("BATCH PROCESSING SUMMARY")
    print("=" * 70)
    print(f"Total expressions:  {len(expressions)}")
    print(f"Successful:         {len(results)}
({len(results)/len(expressions)*100:.1f}%)")
```

```
 400      print(f"Failed:             {len(errors)}
({len(errors)/len(expressions)*100:.1f}%)")
 401
 402      if errors:
 403          print("\nFailed Expressions:")
 404          for err in errors:
 405              print(f"  [{err['index']}] {err['expression']:20} -
{err['error']}")
 406
 407      print()
 408
 409
 410  def main():
 411      """Run all error handling examples"""
 412      print("\n")
 413      print("╔" + "═" * 68 + "╗")
 414      print("║" + " " * 68 + "║")
 415      print("║" + "  RPN CALCULATOR - ERROR HANDLING
EXAMPLES".center(68) + "║")
 416      print("║" + " " * 68 + "║")
 417      print("╚" + "═" * 68 + "╝")
 418      print("\n")
 419
 420      try:
 421          example_basic_error_handling()
 422          example_invalid_operations()
 423          example_csv_processing_with_errors()
 424          example_graceful_degradation()
 425          example_retry_pattern()
 426          example_validation_pattern()
 427          example_logging_errors()
 428          example_batch_processing()
 429
 430          print("=" * 70)
 431          print("ALL ERROR HANDLING EXAMPLES COMPLETED")
 432          print("=" * 70)
 433          print()
 434          print("Key Takeaways:")
 435          print("  • Always use try-except blocks with
CalculatorError")
 436          print("  • Validate inputs before calculation when possible")
 437          print("  • Provide meaningful error messages to users")
 438          print("  • Use logging for production systems")
 439          print("  • Implement graceful degradation for non-critical
errors")
 440          print("  • Consider retry/fallback patterns for robustness")
 441          print()
 442
 443      except Exception as e:
 444          print(f"\n❌ Unexpected error in examples: {e}")
 445          import traceback
 446          traceback.print_exc()
 447
 448
 449  if __name__ == "__main__":
```

```
450     main()
```